



LAPORAN PRAKTIKUM

Pengembangan Aplikasi Web II

Disusun oleh:

Nama : Dzikri Cahayadi Alamsyah

NPM : 2428240126

Kelas : SI5C

PROGRAM STUDI SISTEM INFORMASI
UNIVERSITAS MULTI DATA PALEMBANG
TAHUN AKADEMIK 2026/2027

Identitas Praktikum

Nama Mahasiswa	Dzikri Cahyadi Alamsyah
NPM	2428240126
Kelas	SI5C
Pertemuan Ke-	3 - 4
Topik / Judul Praktikum	Tugas 1 – RESTful API Murni dengan Express.js (Topik 15 – Musik / Resource Lagu)
Tanggal Pelaksanaan	28/09/2026
Dosen Pengampu	Nur Rachmat, M.Kom.

I. Tujuan Praktikum

1. Mahasiswa mampu membuat backend RESTful API menggunakan Express.js

II. Dasar Teori

RESTful API (Representational State Transfer Application Programming Interface) merupakan pendekatan dalam pengembangan layanan web yang memungkinkan aplikasi saling berkomunikasi menggunakan protokol HTTP. Dalam RESTful API, data direpresentasikan sebagai resource yang dapat diakses melalui endpoint tertentu. Pada praktikum ini digunakan Express.js sebagai framework Node.js untuk membangun API. Routing Express digunakan untuk menghubungkan metode HTTP dengan endpoint tertentu, seperti GET, POST, PUT, dan DELETE.

Metode GET digunakan untuk mengambil data, POST untuk menambahkan data baru, PUT untuk memperbarui data, sedangkan DELETE digunakan untuk menghapus data. Express menyediakan `app.get()`, `app.post()`, `app.put()`, dan `app.delete()` untuk menangani request tersebut.

Express menggunakan middleware `express.json()` untuk memproses request dengan payload JSON sehingga data dapat diakses melalui `req.body`. Response API pada praktikum ini menggunakan format JSON dan menerapkan HTTP status code seperti 200, 201, 400, dan 404.

Resource yang digunakan adalah Lagu dengan atribut id, judul, artis, album, tahunRilis, dan durasiDetik. Data disimpan sementara dalam array JavaScript dan dikelola melalui endpoint `/songs`.

III. Alat dan Bahan

No.	Nama Alat / Software	Versi / Keterangan
1	Node.js	v26.2.0
2	Visual Studio Code	1.139.1
3	Express.js	Framework Node.js untuk RESTful API
4	Thunder Client	Pengujian endpoint API
5	Git / GitHub	Version control dan repository

IV. Langkah Kerja

1. Membuat project baru menggunakan Node.js dengan npm init -y.

2. Menginstal Express.js sebagai dependency dan Nodemon sebagai development dependency.

3. Mengatur package.json dengan app.js sebagai entry point serta script start dan dev.

4. Membuat app.js, menginisialisasi Express, menggunakan express.json(), dan menentukan PORT.

5. Membuat tiga data awal resource Lagu serta variabel nextId.

6. Membuat GET /songs untuk mengambil seluruh lagu dan filter menggunakan ?artis=.

7. Membuat GET /songs/:id untuk mengambil satu lagu berdasarkan ID.

8. Membuat POST /songs dengan validasi field wajib dan tipe data serta ID otomatis.

9. Membuat PUT /songs/:id dengan validasi dan pembaruan field.

10. Membuat DELETE /songs/:id dengan status 200 jika berhasil dan 404 jika data tidak ditemukan.

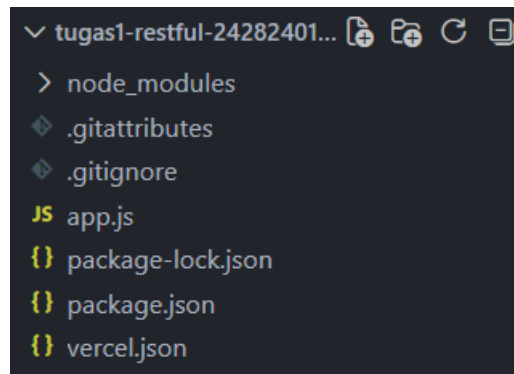
11. Menambahkan middleware catch-all 404 untuk endpoint yang tidak tersedia.

12. Menguji endpoint menggunakan Thunder Client.

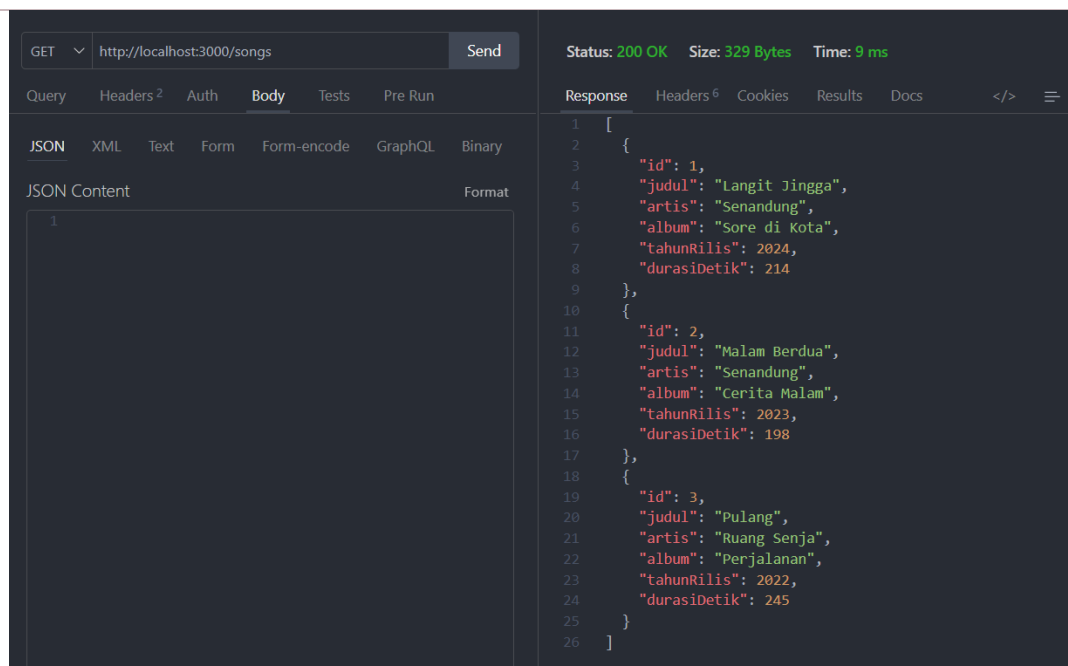
13. Membuat .gitignore dengan isi node_modules/ agar dependency tidak diikutkan ke repository.

14. Menyiapkan repository GitHub dan konfigurasi deployment Vercel.

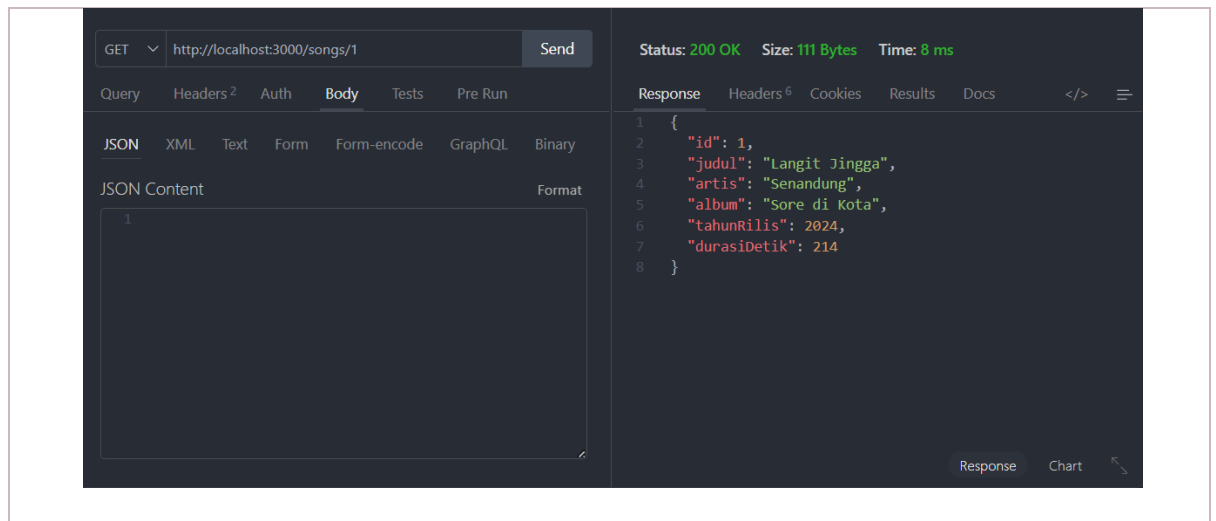
V. Hasil dan Pembahasan



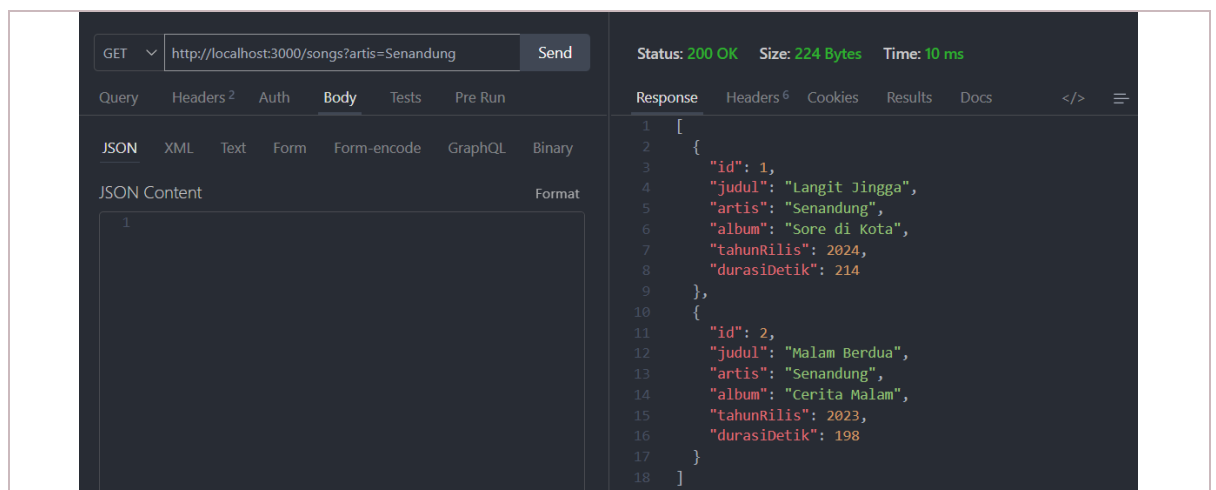
Gambar 1. Struktur project RESTful API Express.js (app.js, package.json, .gitignore).



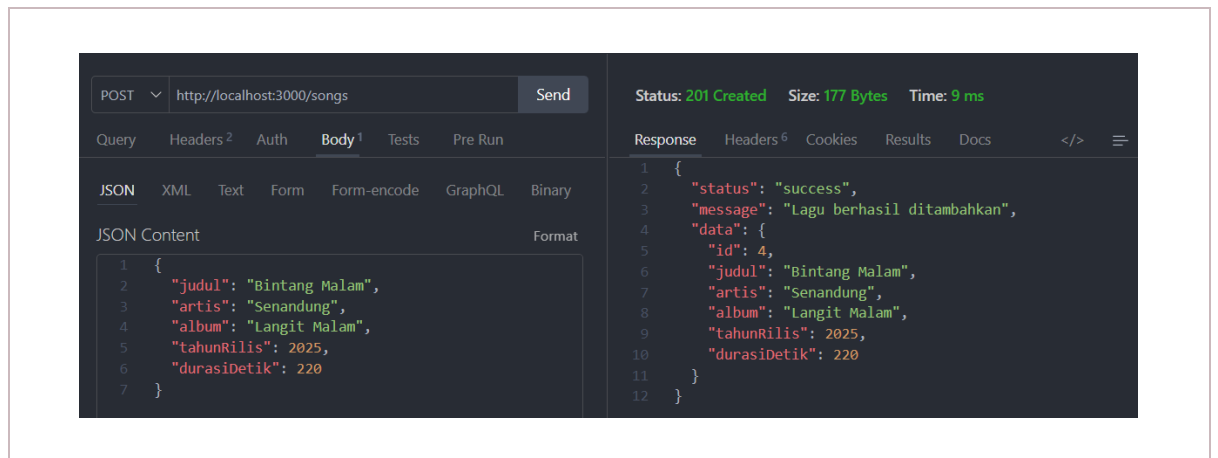
Gambar 2. Pengujian GET /songs: GET http://localhost:3000/songs – mengambil seluruh data lagu.



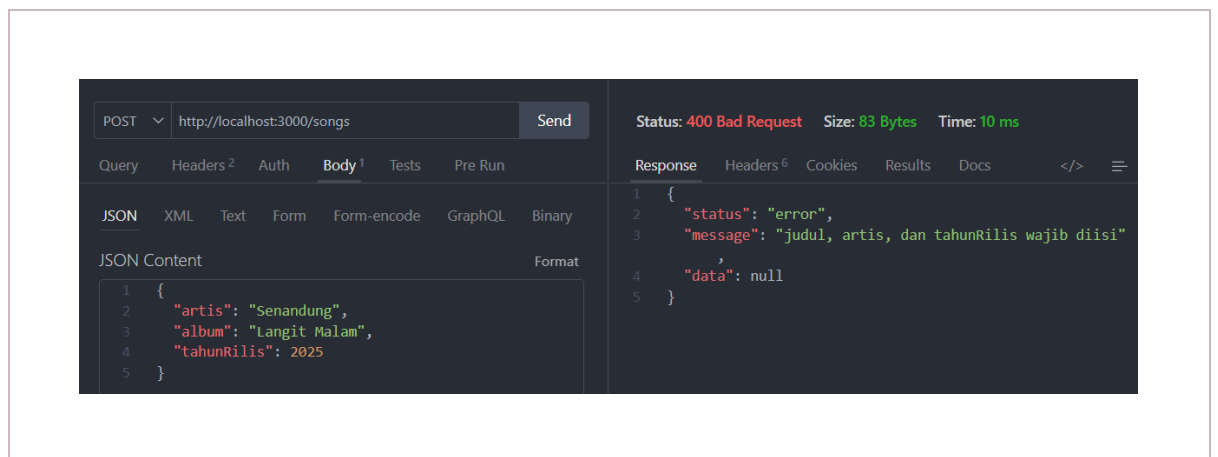
Gambar 3. Pengujian GET /songs/1: GET http://localhost:3000/songs/1 – mengambil lagu berdasarkan ID.



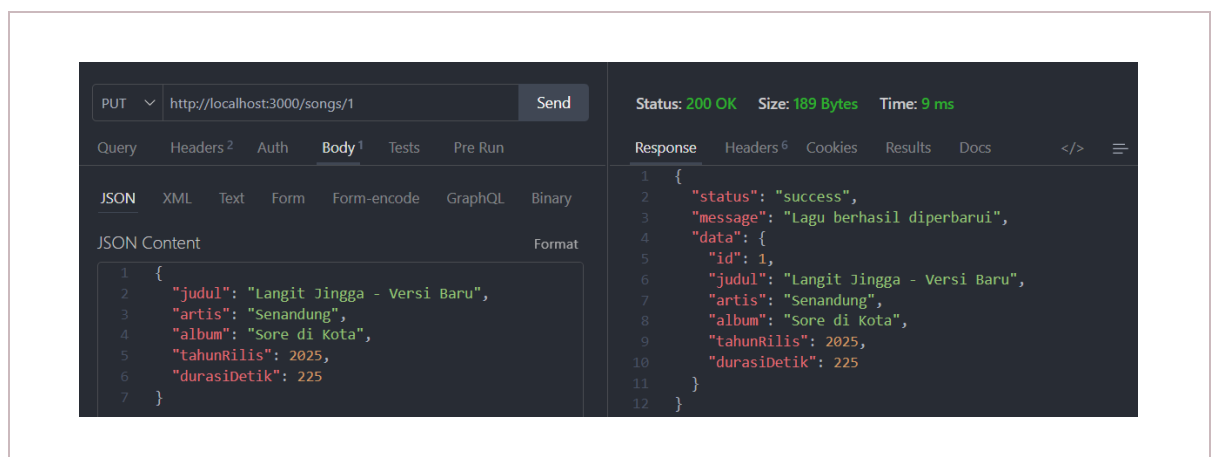
Gambar 4. Pengujian GET filter: GET http://localhost:3000/songs?artis=Senandung – memfilter lagu berdasarkan artis.



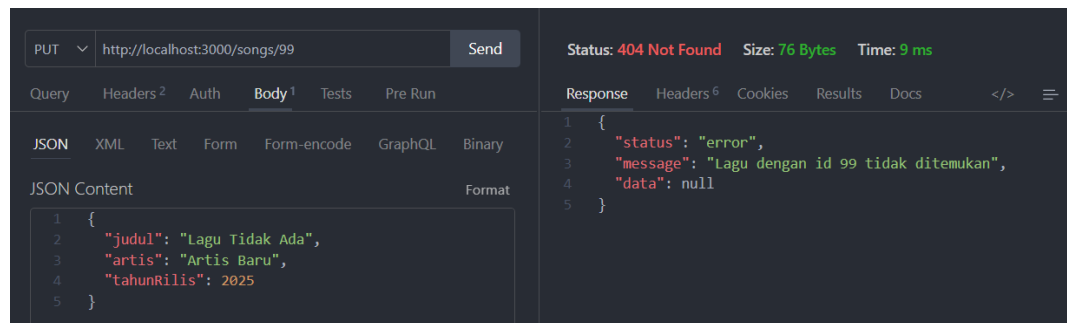
Gambar 5. Pengujian POST /songs: POST http://localhost:3000/songs – menambahkan lagu baru dengan body lengkap.



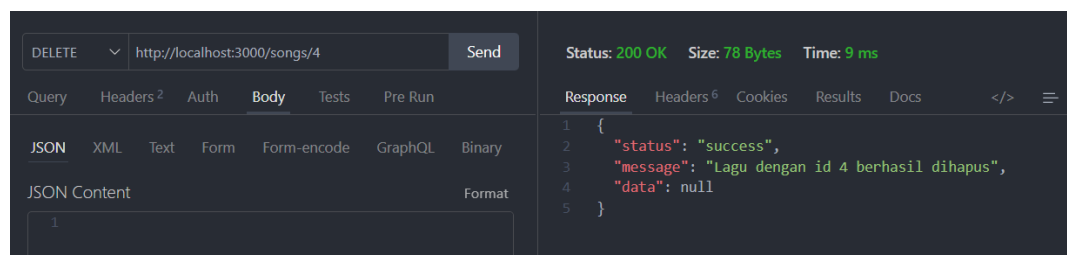
Gambar 6. Pengujian POST invalid: POST http://localhost:3000/songs – menguji field wajib kosong.



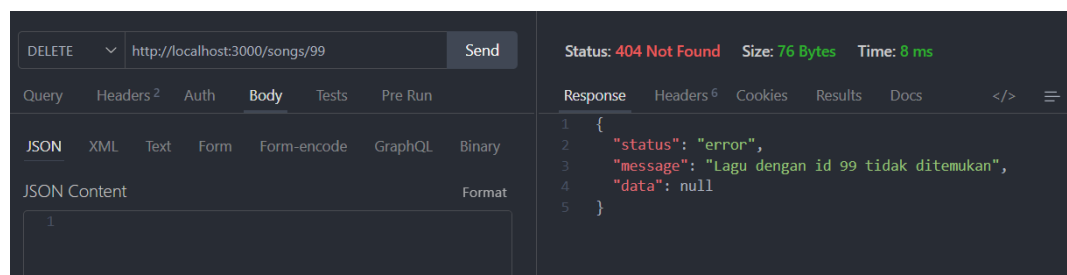
Gambar 7. Pengujian PUT /songs/1: PUT http://localhost:3000/songs/1 – memperbarui lagu ID 1.



Gambar 8. Pengujian PUT /songs/99: PUT http://localhost:3000/songs/99 – menguji ID tidak ditemukan.



Gambar 9. Pengujian DELETE /songs/1: DELETE http://localhost:3000/songs/4 – menghapus lagu ID 4.



Gambar 10. Pengujian DELETE /songs/99: DELETE http://localhost:3000/songs/99 – menguji penghapusan ID tidak ditemukan.

Pembahasan:

Project menggunakan `app.js` sebagai file utama, `package.json` untuk konfigurasi project dan dependency, serta `.gitignore` untuk mengabaikan `node_modules` (Gambar 1). `GET /songs` mengambil seluruh data lagu, `GET /songs/:id` mengambil satu lagu berdasarkan ID, dan parameter `?artis=` memfilter lagu berdasarkan artis, dengan status yang diharapkan 200 (Gambar 2–4). Skenario `GET /songs/99` (ID tidak ditemukan) diharapkan menghasilkan status 404.

`POST /songs` menambahkan lagu baru dengan body `{"judul": "Bintang Malam", "artis": "Senandung", "album": "Langit Malam", "tahunRilis": 2025, "durasiDetik": 220}` dan status yang diharapkan 201, sedangkan field wajib kosong diharapkan menghasilkan status 400 (Gambar 5–6). `PUT /songs/:id` memperbarui data lagu dengan validasi, status yang diharapkan 200 jika berhasil dan 404 jika ID tidak ditemukan (Gambar 7–8).

`DELETE /songs/:id` menghapus lagu dengan status 200 jika berhasil dan 404 jika data tidak ditemukan (Gambar 9–10). Hasil aktual pengujian setiap endpoint: **[[HARUS DIISI SENDIRI]]**

VI. Kendala dan Solusi

No.	Kendala	Solusi
1	PUT /songs/:id sempat menghasilkan Cannot PUT /songs/999.	Memindahkan route app.put() agar berada di luar blok app.post().
2	Format response awal belum sesuai ketentuan tugas.	Menyesuaikan response GET dan menambahkan status, message, dan data.
3	Folder node_modules perlu dicegah masuk repository.	Membuat .gitignore dengan isi node_modules/.
4	Project dikerjakan bertahap menggunakan Git.	Perubahan utama dibuat dalam commit terpisah agar riwayat dapat dilacak.

VII. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, RESTful API berhasil dikembangkan menggunakan Node.js dan Express.js dengan resource songs. API menyediakan endpoint GET, POST, PUT, dan DELETE untuk mengelola data lagu.

Implementasi menerapkan validasi input, HTTP status code, response JSON, filter query parameter, serta penanganan data dan endpoint yang tidak ditemukan.

Pengujian dilakukan menggunakan Thunder Client, dan source code dikelola dengan Git serta .gitignore.

Daftar Pustaka

Express.js. (n.d.). *Express - Node.js web application framework*. Retrieved September 28, 2026, from <https://expressjs.com/>

Git. (n.d.). *Git documentation*. Retrieved September 28, 2026, from <https://git-scm.com/doc>

Node.js. (n.d.). *Node.js documentation*. Retrieved September 28, 2026, from <https://nodejs.org/docs/latest/api/>

Vercel. (n.d.). *Vercel documentation*. Retrieved September 28, 2026, from <https://vercel.com/docs>

Lampiran

Tautan Repository GitHub:

<https://github.com/DzikriMDP/tugas1-restful-2428240126>

Tautan Deployment (jika ada):

<https://tugas1-restful-2428240126.vercel.app>